

APPENDIX

A TAXONOMY OF HALLUCINATIONS IN CODE GENERATION

A.1 Codebook of Code Hallucinations

In order to present the hallucination of each category code more intuitively, we summarize a codebook based on our pilot analysis. The codebook includes an example and explanation for each hallucination cause, type, and impact. Additionally, we provide qualitative definitions and a reference example for each level of text complexity in the prompt.

A.1.1 Hallucination Categories.

I. Behavior Conflicting

- **Dataset:** CoderEval (Python)
- **Example:**

```
1 def _replace_register(flow_params, register_number, register_value):
2     """
3     Replace value from flows to given register number
4
5     'register_value' key in dictionary will be replaced by register number
6     given by 'register_number'
7
8     :param flow_params: Dictionary containing defined flows
9     :param register_number: The number of register where value will be stored
10    :param register_value: Key to be replaced by register number
11    """
12
13    # Hallucinatory code
14    if register_number in flow_params:
15        flow_params[register_number] = register_value
16    else:
17        print(f"Register_number_{register_number}_not_found_in_the_flow_
18            parameters.")
19
20    # Reference code
21    try:
22        reg_port = flow_params[register_value]
23        del flow_params[register_value]
24        flow_params['reg{:d}'].format(register_number) = reg_port
25    except KeyError:
26        pass
27    return flow_params # expose the observer to the test_module
```

- **Explanation:** The function description clearly states that register_value is the **key** of the dictionary, but the hallucinatory code uses it as the **value** of the dictionary, which conflicts with the requirement.

II. Data Conflicting

- **Dataset:** HumanEval
- **Example:**

```
1 def fib4(n: int):
```

```

2      """The Fib4 number sequence is a sequence similar to the Fibonacci
      sequeence that's defined as follows:
3      fib4(0) -> 0
4      fib4(1) -> 0
5      fib4(2) -> 2
6      fib4(3) -> 0
7      fib4(n) -> fib4(n-1) + fib4(n-2) + fib4(n-3) + fib4(n-4).
8      Please write a function to efficiently compute the n-th element of the
      fib4 number sequence. Do not use recursion.
9      >>> fib4(5)
10     4
11     >>> fib4(6)
12     8
13     >>> fib4(7)
14     14
15     """
16
17     # Hallucinatory code
18     if n <= 3:
19         return 0
20     else:
21         fib = [0, 0, 0, 0]
22         for i in range(4, n + 1):
23             fib.append(fib[i - 1] + fib[i - 2] + fib[i - 3] + fib[i - 4])
24         return fib[n]
25
26     # Reference code
27     results = [0, 0, 2, 0]
28     if n < 4:
29         return results[n]
30     for _ in range(4, n + 1):
31         results.append(results[-1] + results[-2] + results[-3] + results[-4])
32         results.pop(0)
33     return results[-1]
34

```

- **Explanation:** The requirement explicitly states that `fib4(2)` is the constant 2, but the hallucinated code incorrectly treats `fib4(2)` as 0, causing a direct semantic conflict between the code and the constant specified in the requirement.

III. Undefined Variables

- **Dataset:** HumanEval
- **Example:**

```

1  def is_simple_power(x, n):
2      """Your task is to write a function that returns true if a number x is a
      simple
3      power of n and false in other cases.
4      x is a simple power of n if n**int=x
5      For example:
6      is_simple_power(1, 4) => true
7      is_simple_power(2, 2) => true
8      is_simple_power(8, 2) => true

```

```

9     is_simple_power(3, 2) => false
10    is_simple_power(3, 1) => false
11    is_simple_power(5, 3) => false
12    """
13
14    # Hallucinatory code
15    return x == n ** int(log(x, n))
16
17    # Reference code
18    if (n == 1):
19        return (x == 1)
20    power = 1
21    while (power < x):
22        power = power * n
23    return (power == x)
24

```

- **Explanation:** The log function is not defined.

IV. Useless Statements(unexecuted)

- **Dataset:** HumanEval
- **Example:**

```

1  def minSubArraySum(nums):
2      """
3      Given an array of integers nums, find the minimum sum of any non-empty
4      sub-array
5      of nums.
6      Example
7      minSubArraySum([2, 3, 4, 1, 2, 4]) == 1
8      minSubArraySum([-1, -2, -3]) == -6
9      """
10
11     # Hallucinatory code
12     if not nums:
13         return 0
14
15     min_sum = float('inf')
16     window_sum = 0
17
18     for i in range(len(nums)):
19         window_sum += nums[i]
20         if window_sum < min_sum:
21             min_sum = window_sum
22         window_sum -= nums[i - len(nums) + 1] if i >= len(nums) else 0
23
24     return min_sum
25
26     # Reference code
27     max_sum = 0
28     s = 0
29     for num in nums:
30         s += -num

```

```

30         if (s < 0):
31             s = 0
32             max_sum = max(s, max_sum)
33         if max_sum == 0:
34             max_sum = max(-i for i in nums)
35         min_sum = -max_sum
36         return min_sum
37

```

- **Explanation:** Since the loop condition is for `i in range(len(nums))`, `i >= len(nums)` is always false, so `-= nums[i - len(nums) + 1]` cannot be executed.

V. Useless Statements(executed without effect)

- **Dataset:** HumanEval
- **Example:**

```

1  from typing import List
2
3
4  def parse_nested_parens(paren_string: str) -> List[int]:
5      """ Input to this function is a string represented multiple groups for
6          nested parentheses separated by spaces.
7          For each of the group, output the deepest level of nesting of parentheses
8          .
9          E.g. (()()) has maximum two levels of nesting while (((()))) has three.
10
11      >>> parse_nested_parens('(()()) (((()))) () (((()))())')
12      [2, 3, 1, 3]
13      """
14
15      # Hallucinatory code
16      depths = []
17      for group in paren_string.split():
18          depth = 0
19          max_depth = 0
20          stack = []
21          for char in group:
22              if char == '(':
23                  depth += 1
24                  stack.append(char)
25              elif char == ')':
26                  depth -= 1
27                  stack.pop()
28              max_depth = max(max_depth, depth)
29          depths.append(max_depth)
30      return depths
31
32      # Reference code
33      def parse_paren_group(s):
34          depth = 0
35          max_depth = 0
36          for c in s:
37              if c == '(':

```

```

36         depth += 1
37         max_depth = max(depth, max_depth)
38     else:
39         depth -= 1
40
41     return max_depth
42
43     return [parse_paren_group(x) for x in paren_string.split('_') if x]
44

```

- **Explanation:** The stack variable in the hallucinatory code has only been written but not read, so these statements have no impact on functionality of the function.

VI. Inconsistent Libraries

- **Dataset:** CodeEval (Python)
- **Example:**

```

1  def is_local(host):
2      """
3      Checks if the host is the localhost
4
5      :param host: The hostname or ip
6      :return: True if the host is the localhost
7      """
8
9      # Hallucinatory code
10     import scapy
11
12     return host in ["127.0.0.1", "localhost", scapy.gethostname()]
13
14     # Reference code
15     import socket
16
17     return host in ["127.0.0.1",
18                    "localhost",
19                    socket.gethostname(),
20                    # just in case socket.gethostname() does not work we
21                    also try the following:
22                    platform.node(),
23                    socket.gethostbyaddr(socket.gethostname())[0]
24

```

- **Explanation:** The hallucinatory code confuses the scapy library and socket library, resulting in the library name being scapy but the method name being gethostname in the socket library when obtaining the host name. The method that scapy should use is `scapy.all.gethostbyaddr()`. In a sense, this can also be seen as a *library knowledge hallucination*, but since LLM only confuses two libraries during generation, rather than lacking accurate knowledge of these two libraries, it is more appropriate to classify it as *inconsistent libraries*.

VII. Fragmented Logics

- **Dataset:** HumanEval
- **Example:**

```

1 def skjksdkd(lst):
2     """You are given a list of integers.
3     You need to find the largest prime value and return the sum of its digits
4     .
5     Examples:
6     For lst = [0,3,2,1,3,5,7,4,5,5,5,2,181,32,4,32,3,2,32,324,4,3] the output
7         should be 10
8     For lst = [1,0,1,8,2,4597,2,1,3,40,1,2,1,2,4,2,5,1] the output should be
9         25
10    For lst = [1,3,1,32,5107,34,83278,109,163,23,2323,32,30,1,9,3] the output
11        should be 13
12    For lst = [0,724,32,71,99,32,6,0,5,91,83,0,5,6] the output should be 11
13    For lst = [0,81,12,3,1,21] the output should be 3
14    For lst = [0,8,1,2,1,7] the output should be 7
15    """
16
17    # Hallucinatory code
18    # Find the largest prime value in the list
19    largest_prime = None
20    for i in range(len(lst)):
21        if is_prime(lst[i]):
22            largest_prime = lst[i]
23            break
24
25    # Return the sum of the digits of the largest prime value
26    if largest_prime is not None:
27        return sum(map(int, str(largest_prime)))
28    else:
29        return 0
30
31    # Reference code
32    def isPrime(n):
33        for i in range(2, int(n**0.5)+1):
34            if n%i==0:
35                return False
36        return True
37
38    maxx = 0
39    i = 0
40    while i < len(lst):
41        if(lst[i] > maxx and isPrime(lst[i])):
42            maxx = lst[i]
43            i+=1
44    result = sum(int(digit) for digit in str(maxx))
45    return result

```

- **Explanation:** In the statement highlighted in red in the hallucinatory code, LLM initially names the variable `largest_prime`, indicating that LLM's logic at this point is to find the **largest** prime number, which is consistent with the requirement. But in the subsequent

statements, the logic of LLM becomes to search for the **first** prime number in the list, which is disconnected from the previous logic. It should be noted that the latter half of the highlighted code can actually be considered a *functional requirement conflicting*, but it is more accurate to classify it as *fragmented logics* because the naming of variable `largest_prime` proves that LLM did not initially misunderstand the requirement, but only forgot or confused it during the generation process.

VIII. Library/Project

- **Dataset:** CoderEval (Java)
- **Example:**

```

1  /**
2   * Finds a minimum lower bound for every key.
3   * @param keys a list of keys.
4   * @return the computed key upper bound.
5   */
6
7  private List<Integer> computeUpperBounds(List<K> keys){
8
9      // Hallucinatory code
10     List<Integer> upperBounds = new ArrayList<>();
11     for(Integer key : keys){
12         upperBounds.add(key + 1);
13     }
14     return upperBounds;
15
16     // Reference code
17     List<Integer> keyUpperBounds=new ArrayList<>(keys.size());
18     for ( K key : keys) {
19         int upperBound=Integer.MAX_VALUE;
20         for (    Function<K,Integer> upperBoundFunction : upperBounds) {
21             upperBound=Math.min(upperBound,upperBoundFunction.apply(key));
22         }
23         keyUpperBounds.add(upperBound);
24     }
25     return keyUpperBounds;
26 }
27

```

- **Explanation:** Due to LLM's lack of knowledge of open-source project's source code, it is not aware of the existing attributes/methods within the class when generating code, which conflicts with project knowledge.

IX. Algorithm

- **Dataset:** HumanEval
- **Example:**

```

1  import math
2
3
4  def poly(xs: list, x: float):
5      """
6      Evaluates polynomial with coefficients xs at point x.

```

```

7     return xs[0] + xs[1] * x + xs[1] * x^2 + ... xs[n] * x^n
8     """
9     return sum([coeff * math.pow(x, i) for i, coeff in enumerate(xs)])
10
11
12 def find_zero(xs: list):
13     """ xs are coefficients of a polynomial.
14     find_zero find x such that poly(x) = 0.
15     find_zero returns only only zero point, even if there are many.
16     Moreover, find_zero only takes list xs having even number of coefficients
17     and largest non zero coefficient as it guarantees
18     a solution.
19     >>> round(find_zero([1, 2]), 2) # f(x) = 1 + 2x
20     -0.5
21     >>> round(find_zero([-6, 11, -6, 1]), 2) # (x - 1) * (x - 2) * (x - 3) =
22     -6 + 11x - 6x^2 + x^3
23     1.0
24     """
25
26     # Hallucinatory code
27     if len(xs) % 2 != 0:
28         raise ValueError("The_list_of_coefficients_must_have_an_even_number_
29         of_elements.")
30
31     # Start with an initial guess, which is the reciprocal of the leading
32     coefficient
33     x0 = 1.0 / xs[0]
34
35     # Newton-Raphson iteration
36     for _ in range(1000): # Limit the number of iterations to prevent
37         infinite loops
38         # Compute the derivative of the polynomial at x0
39         derivative = sum([i * coeff * math.pow(x0, i) for i, coeff in
40         enumerate(xs)])
41
42         # Check if the derivative is zero to prevent division by zero
43         if derivative == 0:
44             raise ValueError("The derivative of the polynomial is zero,
45             which means the polynomial has a constant term.")
46
47         # Compute the next approximation
48         x1 = x0 - poly(xs, x0) / derivative
49
50         # Check for convergence
51         if abs(x1 - x0) < 1e-10: #
52
53     # Reference code
54     begin, end = -1., 1.
55     while poly(xs, begin) * poly(xs, end) > 0:
56         begin *= 2.0
57         end *= 2.0

```

```

53     while end - begin > 1e-10:
54         center = (begin + end) / 2.0
55         if poly(xs, center) * poly(xs, begin) > 0:
56             begin = center
57         else:
58             end = center
59     return begin
60

```

- **Explanation:** The hallucinatory code uses the Newton-Raphson Method to solve for the zero point of the function, but according to this algorithm, x_0 should be re-selected for iteration when the derivative value is 0 instead of reporting an error. Strictly speaking, this algorithm belongs to the category of mathematics, but because its iterative operations are more suitable for computer calculations, we consider it a computer algorithm.

X. Computer Theory

- **Dataset:** HumanEval
- **Example:**

```

1  from typing import List
2
3
4  def string_xor(a: str, b: str) -> str:
5      """ Input are two strings a and b consisting only of 1s and 0s.
6          Perform binary XOR on these inputs and return result also as a string.
7          >>> string_xor('010', '110')
8              '100'
9          """
10
11     # Hallucinatory code
12     result = ""
13     for i in range(len(a)):
14         if a[i] == "1" and b[i] == "1":
15             result += "0"
16         else:
17             result += "1"
18     return result
19
20     # Reference code
21     def xor(i, j):
22         if i == j:
23             return '0'
24         else:
25             return '1'
26
27     return ''.join(xor(x, y) for x, y in zip(a, b))
28

```

- **Explanation:** The highlighted code assumes that only when a and b are both 1, their XOR value is 0, which conflicts with the XOR calculation rule in computer theory.

XI. Common Sense

- **Dataset:** HumanEval

- **Example:**

```

1  def encode(message):
2      """
3      Write a function that takes a message, and encodes in such a
4      way that it swaps case of all letters, replaces all vowels in
5      the message with the letter that appears 2 places ahead of that
6      vowel in the english alphabet.
7      Assume only letters.
8
9      Examples:
10     >>> encode('test')
11     'TGST'
12     >>> encode('This is a message')
13     'tHKS KS C MGSSCGG'
14     """
15
16     # Hallucinatory code
17     vowels = 'aeioru'
18     encoded_message = ''
19     for char in message:
20         if char.lower() in vowels:
21             index = vowels.index(char.lower())
22             if char.isupper():
23                 encoded_message += chr(ord('A') + (index + 2) % 26)
24             else:
25                 encoded_message += chr(ord('a') + (index + 2) % 26)
26         else:
27             encoded_message += char
28     return encoded_message
29
30     # Reference code
31     vowels = "aeiouAEIOU"
32     vowels_replace = dict([(i, chr(ord(i) + 2)) for i in vowels])
33     message = message.swapcase()
34     return ''.join([vowels_replace[i] if i in vowels else i for i in message
35 ])
36

```

- **Explanation:** Common sense tells us that vowels include “aeiou”, but the hallucinatory code has an additional “r”, which conflicts with common sense.

XII. Mathematics & Natural Science

- **Dataset:** HumanEval
- **Example:**

```

1  def derivative(xs: list):
2      """ xs represent coefficients of a polynomial.
3      xs[0] + xs[1] * x + xs[2] * x^2 + ....
4      Return derivative of this polynomial in the same form.
5      >>> derivative([3, 1, 2, 4, 5])
6      [1, 4, 12, 20]
7      >>> derivative([1, 2, 3])

```

```

8     [2, 6]
9     """
10
11     # Hallucinatory code
12     if len(xs) == 1:
13         return [0]
14     else:
15         return [i * xs[idx + 1] for idx, i in enumerate(xs[1:])]
16
17     # Reference code
18     return [(i * x) for i, x in enumerate(xs)][1:]
19

```

- **Explanation:** The formula for calculating the derivative of a polynomial function is incorrect.

A.1.2 Hallucination Causes.

I. Ambiguous or Incomplete

- **Dataset:** HumanEval
- **Example:**

```

1  def pairs_sum_to_zero(l):
2      """
3      pairs_sum_to_zero takes a list of integers as an input.
4      it returns True if there are two distinct elements in the list that
5      sum to zero, and False otherwise.
6      >>> pairs_sum_to_zero([1, 3, 5, 0])
7      False
8      >>> pairs_sum_to_zero([1, 3, -2, 1])
9      False
10     >>> pairs_sum_to_zero([1, 2, 3, 7])
11     False
12     >>> pairs_sum_to_zero([2, 4, -5, 3, 5, 7])
13     True
14     >>> pairs_sum_to_zero([1])
15     False
16     """
17
18     # Hallucinatory code
19     if len(l) < 2:
20         return False
21     else:
22         return any(x + y == 0 for x in l for y in l if x != y)
23
24     # Reference code
25     for i, l1 in enumerate(l):
26         for j in range(i + 1, len(l)):
27             if l1 + l[j] == 0:
28                 return True
29     return False
30

```

- **Explanation:** “Two distinct elements” can be interpreted as two elements with **different indices** in the list or as two elements with **different values**. Therefore, this phrasing is

ambiguous, and the LLM chose the incorrect interpretation, resulting in a conflict with the requirements.

II. Lack of Domain-Specific Knowledge

- **Dataset:** CoderEval (Python)
- **Example:**

```

1  def index(self, key):
2      """ Return the index of the given item.
3
4      :param key: a key
5      :return: index
6      :rtype: int
7      """
8
9      # Hallucinatory code
10     for i in range(len(self.keys)):
11         if self.keys[i] == key:
12             return i
13     raise ValueError(f"{repr(key)}_is_not_in_list")
14
15     # Reference code
16     if isinstance(key, int):
17         if 0 <= key < len(self.__keys):
18             return key
19         raise IndexError(key)
20     elif isinstance(key, str):
21         try:
22             return self.__keys.index(key)
23         except ValueError:
24             raise KeyError(key)
25     else:
26         raise TypeError(key)
27

```

- **Explanation:** In the correct code implementation, the function uses the `__key` attribute of the class. However, this project-specific knowledge was not provided in the problem description, leading the LLM to infer a non-existent attribute (`self.key`), causing a conflict between the code and real-world knowledge.

III. Model-related Causes

- **Dataset:** HumanEval
- **Example:**

```

1  def circular_shift(x, shift):
2      """Circular shift the digits of the integer x, shift the digits right by
3      shift
4      and return the result as a string.
5      If shift > number of digits, return digits reversed.
6      >>> circular_shift(12, 1)
7      "21"
8      >>> circular_shift(12, 2)
9      "12"

```

```

9      """
10
11     # Hallucinatory code
12     # Convert the integer to a string
13     str_x = str(x)
14     # If shift is greater than the length of the string, reverse the string
15     if shift >= len(str_x):
16         return str_x[::-1]
17     # Otherwise, shift the string
18     return str_x[-shift:] + str_x[:-shift]
19
20     # Reference code
21     s = str(x)
22     if shift > len(s):
23         return s[::-1]
24     else:
25         return s[len(s) - shift:] + s[:len(s) - shift]
26

```

- **Explanation:** In this problem, the requirement description is complete and clear. However, the conditional statement in the code ($\geq$) conflicts with the condition specified in the requirement ($>$). Therefore, we infer that this hallucination stems from model-related causes.

A.1.3 Hallucination Impacts.

I. Incorrect Functionality

- **Dataset:** HumanEval
- **Example:**

```

1  def popitem(self):
2      """Remove and return the `(key, value)` pair first inserted."""
3
4      # Hallucinatory code
5      if not self.order:
6          raise KeyError("popitem():_dictionary_is_empty")
7      key = self.order.pop(0)
8      value = self.data.pop(key)
9      return (key, value)
10
11     # Reference code
12     try:
13         key = next(iter(self.__order__))
14     except StopIteration:
15         raise KeyError('%s_is_empty' % type(self).__name__) from None
16     else:
17         return (key, self.pop(key))
18

```

- **Explanation:** The hallucinatory code incorrectly uses a non-existent attribute (`order`), whereas the correct attribute should be `__order`, resulting in a semantic conflict with project-specific knowledge. This hallucination causes an `AttributeError` during execution, affecting the functionality of the code.

II. Low Efficiency

- **Dataset:** HumanEval
- **Example:**

```

1  from typing import List
2
3
4  def parse_nested_parens(paren_string: str) -> List[int]:
5      """ Input to this function is a string represented multiple groups for
6          nested parentheses separated by spaces.
7          For each of the group, output the deepest level of nesting of parentheses
8          .
9          E.g. (()()) has maximum two levels of nesting while (((()))) has three.
10
11      >>> parse_nested_parens('(()()) ((())) () ((())()())')
12      [2, 3, 1, 3]
13      """
14
15      # Hallucinatory code
16      depths = []
17      for group in paren_string.split():
18          depth = 0
19          max_depth = 0
20          stack = []
21          for char in group:
22              if char == '(':
23                  depth += 1
24                  stack.append(char)
25              elif char == ')':
26                  depth -= 1
27                  stack.pop()
28              max_depth = max(max_depth, depth)
29          depths.append(max_depth)
30      return depths
31
32      # Reference code
33      def parse_paren_group(s):
34          depth = 0
35          max_depth = 0
36          for c in s:
37              if c == '(':
38                  depth += 1
39                  max_depth = max(depth, max_depth)
40              else:
41                  depth -= 1
42
43          return max_depth
44
45      return [parse_paren_group(x) for x in paren_string.split(' ') if x]

```

- **Explanation:** In the hallucinatory code, the stack variable is written but never read, meaning it has no impact on the function's functionality. This belongs to the category of hallucinations involving *Useless Statements*. Such extraneous statements may consume additional computational resources, potentially leading to decreased code efficiency.

III. Poor Readability

- **Dataset:** HumanEval
- **Example:**

```

1  def valid_date(date):
2      """You have to write a function which validates a given date string and
3      returns True if the date is valid otherwise False.
4      The date is valid if all of the following rules are satisfied:
5      1. The date string is not empty.
6      2. The number of days is not less than 1 or higher than 31 days for
       months 1,3,5,7,8,10,12. And the number of days is not less than 1 or
       higher than 30 days for months 4,6,9,11. And, the number of days is not
       less than 1 or higher than 29 for the month 2.
7      3. The months should not be less than 1 or higher than 12.
8      4. The date should be in the format: mm-dd-yyyy
9
10     for example:
11     valid_date('03-11-2000') => True
12
13     valid_date('15-01-2012') => False
14
15     valid_date('04-0-2040') => False
16
17     valid_date('06-04-2020') => True
18
19     valid_date('06/04/2020') => False
20     """
21
22     # Hallucinatory code
23     if not date:
24         return False
25
26     date_parts = date.split('-')
27     if len(date_parts) != 3:
28         return False
29
30     month, day, year = date_parts
31     if not month.isdigit() or not day.isdigit() or not year.isdigit():
32         return False
33
34     month = int(month)
35     day = int(day)
36     year = int(year)
37
38     if month < 1 or month > 12:
39         return False
40

```

```

41     if day < 1 or day > 31:
42         return False
43
44     if month in [4, 6, 9, 11] and day > 30:
45         return False
46
47     if month == 2 and day > 29:
48         return False
49
50     if month in [1, 3, 5, 7, 8, 10, 12] and day > 31:
51         return False
52
53     return True
54
55     # Reference code
56     try:
57         date = date.strip()
58         month, day, year = date.split('-')
59         month, day, year = int(month), int(day), int(year)
60         if month < 1 or month > 12:
61             return False
62         if month in [1, 3, 5, 7, 8, 10, 12] and day < 1 or day > 31:
63             return False
64         if month in [4, 6, 9, 11] and day < 1 or day > 30:
65             return False
66         if month == 2 and day < 1 or day > 29:
67             return False
68     except:
69         return False
70
71     return True
72

```

- **Explanation:** In the hallucinatory code, when `day > 31`, the code immediately returns False at the first highlighted position. However, at the second highlighted position, the condition `day > 31` is redundantly checked again, making the code within this branch unreachable. This falls into the *Useless Statements* hallucination category. Such illogical redundancy may confuse readers and reduce the overall readability of the code.

IV. Poor Maintainability and Scalability

- **Dataset:** CoderEval (Python)
- **Example:**

```

1 def parse(self, timestr, default=None,
2
3         ignoretz=False, tzinfos=None, **kwargs):
4     """
5     Parse the date/time string into a :class:`datetime.datetime` object.
6
7     :param timestr:
8         Any date/time string using the supported formats.
9
10    :param default:

```

```

11         The default datetime object, if this is a datetime object and not
12         ``None``, elements specified in ``timestr`` replace elements in the
13         default object.
14
15     :param ignoretz:
16         If set ``True``, time zones in parsed strings are ignored and a
17         naive :class:`~datetime.datetime` object is returned.
18
19     :param tzinfos:
20         Additional time zone names / aliases which may be present in the
21         string. This argument maps time zone names (and optionally offsets
22         from those time zones) to time zones. This parameter can be a
23         dictionary with timezone aliases mapping time zone names to time
24         zones or a function taking two parameters (``tzname`` and
25         ``tzoffset``) and returning a time zone.
26
27         The timezones to which the names are mapped can be an integer
28         offset from UTC in seconds or a :class:`~tzinfo` object.
29
30     .. doctest::
31         :options: +NORMALIZE_WHITESPACE
32
33         >>> from dateutil.parser import parse
34         >>> from dateutil.tz import gettz
35         >>> tzinfos = {"BRST": -7200, "CST": gettz("America/Chicago")}
36         >>> parse("2012-01-19 17:21:00 BRST", tzinfos=tzinfos)
37         datetime.datetime(2012, 1, 19, 17, 21, tzinfo=tzoffset(u'BRST',
38         -7200))
39
40         >>> parse("2012-01-19 17:21:00 CST", tzinfos=tzinfos)
41         datetime.datetime(2012, 1, 19, 17, 21,
42         tzinfo=tzfile('/usr/share/zoneinfo/America/
43         Chicago'))
44
45         This parameter is ignored if ``ignoretz`` is set.
46
47     :param **kwargs:
48         Keyword arguments as passed to ``_parse()``.
49
50     :return:
51         Returns a :class:`~datetime.datetime` object or, if the
52         ``fuzzy_with_tokens`` option is ``True``, returns a tuple, the
53         first element being a :class:`~datetime.datetime` object, the second
54         a tuple containing the fuzzy tokens.
55
56     :raises ParserError:
57         Raised for invalid or unknown string format, if the provided
58         :class:`~tzinfo` is not in a valid format, or if an invalid date
59         would be created.
60
61     :raises TypeError:
62         Raised for non-string or character stream input.

```

```

60
61 :raises OverflowError:
62     Raised if the parsed date exceeds the largest valid C integer on
63     your system.
64     """
65
66     # Hallucinatory code
67     if not isinstance(timestr, str):
68         raise TypeError("timestr_must_be_a_string")
69
70     fuzzy_with_tokens = kwargs.get('fuzzy_with_tokens', False)
71     result = self._parse(timestr, **kwargs)
72
73     if fuzzy_with_tokens:
74         components, tokens = result
75     else:
76         components = result
77         tokens = None
78
79     merged = {}
80     if default is not None:
81         merged.update({
82             'year': default.year,
83             'month': default.month,
84             'day': default.day,
85             'hour': default.hour,
86             'minute': default.minute,
87             'second': default.second,
88             'microsecond': default.microsecond,
89             'tzinfo': default.tzinfo
90         })
91     merged.update(components)
92
93     required = ['year', 'month', 'day']
94     for field in required:
95         if merged.get(field) is None:
96             raise ParserError(f"Missing_required_field_{field}")
97
98     try:
99         naive_dt = datetime(
100             year=merged['year'],
101             month=merged['month'],
102             day=merged['day'],
103             hour=merged.get('hour', 0),
104             minute=merged.get('minute', 0),
105             second=merged.get('second', 0),
106             microsecond=merged.get('microsecond', 0)
107         )
108     except ValueError as e:
109         raise ParserError(f"Invalid_date_{e}")
110
111     tzinfo = None

```

```

112     if not ignoretz:
113         if 'tzoffset' in components:
114             tzinfo = tzoffset(' ', components['tzoffset'])
115         elif 'tzname' in components:
116             tzname = components['tzname']
117             if tzinfos is None:
118                 raise ParserError(f"Unknown_timezone_{tzname},_no_tzinfos_
provided")
119             try:
120                 if callable(tzinfos):
121                     tzdata = tzinfos(tzname, components.get('tzoffset'))
122                 else:
123                     tzdata = tzinfos[tzname]
124             except KeyError:
125                 raise ParserError(f"Unknown_timezone_{tzname}") from None
126             except TypeError:
127                 raise ValueError("tzinfos_must_be_a_dict_or_callable") from
None
128
129             if isinstance(tzdata, datetime.tzinfo):
130                 tzinfo = tzdata
131             elif isinstance(tzdata, int):
132                 tzinfo = tzoffset(tzname, tzdata)
133             else:
134                 raise ValueError(f"Invalid_tzinfo_type:{type(tzdata)}")
135         else:
136             tzinfo = merged.get('tzinfo')
137
138     if tzinfo is not None:
139         try:
140             dt = naive_dt.replace(tzinfo=tzinfo)
141         except ValueError as e:
142             raise ParserError(f"Invalid_timezone:{e}")
143     else:
144         dt = naive_dt
145
146     if fuzzy_with_tokens:
147         return (dt, tokens)
148     else:
149         return dt
150
151     # Reference code
152     if default is None:
153         default = datetime.datetime.now().replace(hour=0, minute=0,
154                                                    second=0, microsecond=0)
155
156     res, skipped_tokens = self._parse(timestr, **kwargs)
157
158     if res is None:
159         raise ParserError("Unknown_string_format:%s", timestr)
160
161     if len(res) == 0:

```

```

162         raise ParserError("String_does_not_contain_a_date:_%s", timestr)
163
164     try:
165         ret = self._build_naive(res, default)
166     except ValueError as e:
167         six.raise_from(ParserError(e.args[0] + ":%s", timestr), e)
168
169     if not ignoretz:
170         ret = self._build_tzaware(ret, res, tzinfos)
171
172     if kwargs.get('fuzzy_with_tokens', False):
173         return ret, skipped_tokens
174     else:
175         return ret
176

```

- **Explanation:** In this case, the requirement states that the function should only raise three types of exceptions: `ParserError`, `TypeError`, and `OverflowError`. However, in the hallucinatory code, a `ValueError` is introduced, creating a direct semantic conflict with the requirement. This hallucination may lead to additional exception handling in future modifications or extensions of the function, thereby reducing its maintainability and scalability.

A.1.4 Text Complexity of Prompt.

We established the following four criteria for evaluating complexity: whether the prompt (1) necessitates repeated readings to comprehend its intent, (2) involves multiple reasoning steps, (3) requires specialized domain knowledge or newly introduced definitions, or (4) exhibits syntactical complexity (e.g., nested clauses or prepositional phrases).

Level 1. Simple

- **Definition:** This is the lowest level of text complexity, it denotes a straightforward prompt that does not meet any of the criteria.
- **Example:** `CoderEval` (Java)

```

1  /**
2   * Delete's the specified file if it exists
3   */
4
5   protected static void deleteFile(String fileName){
6

```

Level 2. Moderate

- **Definition:** Prompt at this level of text complexity exhibits 1–2 criteria.
- **Example:** `CoderEval` (Java)

```

1  /**
2   * Computes the global separator list of the {@code graph}. More precisely,
3   *   for every edge $e$ in the $G = (V, E)$ computes list of minimal
4   *   separators $S_e$ in the neighborhood of $e$ and then concatenates these
5   *   lists. Note: the result may contain duplicates
6   * @return the list of minimal separators of every edge $e$ in the inspected
7   *   graph
8   */
9   private List<Pair<List<Pair<Integer,Integer>>,E>> computeGlobalSeparatorList
10    () {

```

6

- **Explanation:** This prompt requires the use of the domain-specific concept of the “minimum separator” and involves multiple steps of reasoning and implementation.

Level 3. *Complex*

- **Definition:** This is the highest level of text complexity. Prompt at this complexity level meets 3–4 criteria or has an exceptionally pronounced instance of any single criterion.
- **Example:** HumanEval

```

1  def minPath(grid, k):
2      """
3      Given a grid with N rows and N columns (N >= 2) and a positive integer k,
4      each cell of the grid contains a value. Every integer in the range [1, N
5      * N]
6      inclusive appears exactly once on the cells of the grid.
7
8      You have to find the minimum path of length k in the grid. You can start
9      from any cell, and in each step you can move to any of the neighbor cells
10     ,
11     in other words, you can go to cells which share an edge with you current
12     cell.
13     Please note that a path of length k means visiting exactly k cells (not
14     necessarily distinct).
15     You CANNOT go off the grid.
16     A path A (of length k) is considered less than a path B (of length k) if
17     after making the ordered lists of the values on the cells that A and B go
18     through (let's call them lst_A and lst_B), lst_A is lexicographically
19     less
20     than lst_B, in other words, there exist an integer index i (1 <= i <= k)
21     such that lst_A[i] < lst_B[i] and for any j (1 <= j < i) we have
22     lst_A[j] = lst_B[j].
23     It is guaranteed that the answer is unique.
24     Return an ordered list of the values on the cells that the minimum path
25     go through.
26
27     Examples:
28
29     Input: grid = [ [1,2,3], [4,5,6], [7,8,9]], k = 3
30     Output: [1, 2, 1]
31
32     Input: grid = [ [5,9,3], [4,1,6], [7,8,2]], k = 1
33     Output: [1]
34     """

```

- **Explanation:** This prompt is lengthy and requires repeated readings to fully comprehend. Additionally, it exhibits high complexity in both syntax and reasoning steps.

A.2 Composition of the Prompts Used for Code Generation

Below are examples of the prompt compositions used in HumanEval and CoderEval:

- **HumanEval:**

I want you to play the role of a programmer, and please try to directly implement this function according to its description:

```

1 def minSubArraySum(nums):
2     """
3     Given an array of integers nums, find the minimum sum
4     of any non-empty sub-array of nums.
5     Example
6     minSubArraySum([2, 3, 4, 1, 2, 4]) == 1
7     minSubArraySum([-1, -2, -3]) == -6
8     """

```

- **CoderEval:**

I want you to play the role of a programmer, and please try to directly implement this function according to its description:

```

1 /**
2  * Trim the elements of the given String array, calling <code>String.trim
3  *   </code> on each of them.
4  * @param array the original String array
5  * @return the resulting array (of the same size) with trimmed elements
6  */
7 public static String[] trimArrayElements(String[] array){

```

A.3 Details about Manual Analysis

In the first phase of labeling (the initial 20%), both labelers have over four years of experience in Java and Python programming. In the second phase (the remaining 80%), all the ten labelers possess at least two years of experience in Java or Python programming. During the second phase, we developed an online labeling platform to facilitate labelers in comparing prompts, generated code, and reference code. Additionally, the platform's clear interface helped reduce the likelihood of low-level errors during labeling. The entire labeling and taxonomy construction process costed approximately 300 person-hours.

A.4 Details about Kappa Score

Due to the particularity of our label (label by line), it cannot be directly calculated using formulas. Therefore, we first performed a series of transformations on the label results. For example, to calculate the kappa score of the causes of hallucinations, we first generate a 2D vector of length n (where n is the number of generated code lines), where each element (1D vector) corresponds to the labeled causes of hallucinations contained in the corresponding line. If the line does not contain any hallucination, we use a specific serial number for placeholder. Then, we encode each element in the 2D vector into a 01 vector of length m (where m is the number of cause categories of hallucinations, including the placeholder number). Fig. 1 shows a schematic diagram of label conversion. After this conversion, we obtained a 01 matrix for each code, allowing us to calculate the kappa coefficient on the rows and then average it across the columns. The specific algorithm is as follows:

Here, M_1 and M_2 are the new matrices obtained by concatenating column-wise the 01 matrices of corresponding labeler on each code.

Algorithm 1 Calculate Cohen’s Kappa Score of Labeling Results

Input: Two matrices M_1 and M_2 of the same dimensions, represent the labeling results of two labelers on the same dataset.
Output: The Cohen’s Kappa Score of the two labelers’ labeling results.
 $scores \leftarrow []$
for each row1, row2 in zip(M_1 , M_2) **do**
 $score \leftarrow \text{cohen_kappa_score}(\text{row1}, \text{row2})$
 append $score$ to $scores$
end for
return $\frac{1}{|scores|} \sum scores$

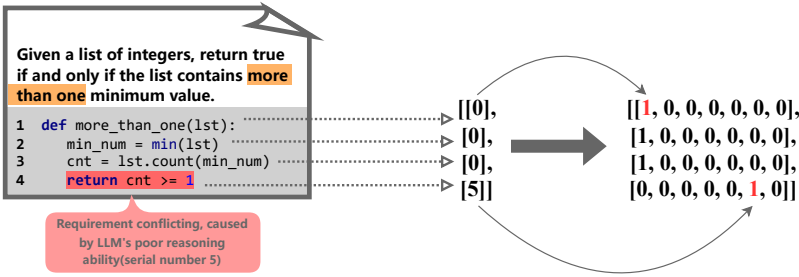


Fig. 1. schematic diagram of label conversion.

The calculating results are shown in Table 1. From the results, it can be seen that all kappa scores are above 0.6, indicating substantial agreement. On this basis, we select the more experienced annotator from each group as the final annotation result.

Table 1. Kappa score between annotators. Two annotators on the same row annotate the same set of samples.

Participants		Kappa Score		
<i>participant A</i>	<i>participant B</i>	<i>Cause</i>	<i>Category</i>	<i>Impact</i>
A1	B1	0.9260	0.9266	0.9273
A2	B2	0.9621	0.9634	0.9631
A3	B3	0.7650	0.7719	0.7893
A4	B4	0.8381	0.8328	0.8431
A5	B5	0.7836	0.7867	0.7889

A.5 Examples of Non-hallucination Code Error Patterns

I. Syntactic Error

• **Example:**

```

1 from typing import List
2
3
4 def string_xor(a: str, b: str) -> str:
5     """ Input are two strings a and b consisting only of 1s and 0s.
6     Perform binary XOR on these inputs and return result also as a string.
7     >>> string_xor('010', '110')
```

```

8     '100'
9     """
10
11     # Code with Syntactic Error
12     def xor(i, j):
13         if i = j:
14             return '0'
15         else:
16             return '1'
17
18     return ''.join(xor(x, y) for x, y in zip(a, b))
19
20     # Reference code
21     def xor(i, j):
22         if i == j:
23             return '0'
24         else:
25             return '1'
26
27     return ''.join(xor(x, y) for x, y in zip(a, b))
28

```

- **Explanation:** The highlighted portion incorrectly treats the assignment operator as the equality operator, resulting in a simple syntax error.

II. Logical Reasoning Error

- **Example:**

```

1  def is_simple_power(x, n):
2      """Your task is to write a function that returns true if a number x is a
3      simple
4      power of n and false in other cases.
5      x is a simple power of n if n**int=x
6      For example:
7      is_simple_power(1, 4) => true
8      is_simple_power(2, 2) => true
9      is_simple_power(8, 2) => true
10     is_simple_power(3, 2) => false
11     is_simple_power(3, 1) => false
12     is_simple_power(5, 3) => false
13     """
14
15     # Code with Logical Reasoning Error
16     if (n == 1):
17         return (x == 1)
18     power = 1
19     while (power <= x):
20         power = power * n
21     return (power == x)
22
23     # Reference code
24     if (n == 1):
25         return (x == 1)

```

```

25     power = 1
26     while (power < x):
27         power = power * n
28     return (power == x)
29

```

- **Explanation:** The highlighted portion uses a "less than or equal to" relationship to determine whether to exit the loop. However, under this logic, when x is a simple power of n , the power variable will become $x * n$ after the loop, causing the return value to be False (when $n \neq 1$), which leads to incorrect functionality. This error may have been caused by faulty logical reasoning in the LLM, but it does not create a direct semantic conflict with the requirements.

III. Incomplete Implementation

- **Example:**

```

1  def skjkasdkd(lst):
2      """You are given a list of integers.
3      You need to find the largest prime value and return the sum of its digits
4      .
5      Examples:
6      For lst = [0,3,2,1,3,5,7,4,5,5,5,2,181,32,4,32,3,2,32,324,4,3] the output
7          should be 10
8      For lst = [1,0,1,8,2,4597,2,1,3,40,1,2,1,2,4,2,5,1] the output should be
9          25
10     For lst = [1,3,1,32,5107,34,83278,109,163,23,2323,32,30,1,9,3] the output
11         should be 13
12     For lst = [0,724,32,71,99,32,6,0,5,91,83,0,5,6] the output should be 11
13     For lst = [0,81,12,3,1,21] the output should be 3
14     For lst = [0,8,1,2,1,7] the output should be 7
15     """
16
17     # Code with Incomplete Implementation
18     def isPrime(n):
19         for i in range(2, int(n**0.5)+1):
20             if n%i==0:
21                 return False
22
23         return True
24
25     maxx = 0
26     i = 0
27     while i < len(lst):
28         if(lst[i] > maxx and isPrime(lst[i])):
29             maxx = lst[i]
30             i+=1
31     return maxx
32
33     # Reference code
34     def isPrime(n):
35         for i in range(2, int(n**0.5)+1):
36             if n%i==0:
37                 return False
38

```

```

35         return True
36     maxx = 0
37     i = 0
38     while i < len(lst):
39         if(lst[i] > maxx and isPrime(lst[i])):
40             maxx = lst[i]
41             i+=1
42     result = sum(int(digit) for digit in str(maxx))
43     return result
44

```

- **Explanation:** In this example, the erroneous code only implements half of the requirement (“find the largest prime value”) while neglecting the subsequent requirement (“return the sum of its digits”). It falls under incomplete implementation of the requirement, but does not cause a direct conflict with the requirements.

IV. Invalid Generation

- **Example:**

```

1  /**
2   * Finds a minimum lower bound for every key.
3   * @param keys a list of keys.
4   * @return the computed key upper bound.
5   */
6
7  private List<Integer> computeUpperBounds(List<K> keys){
8
9      // Code with Invalid Generation
10     List<Integer> keyUpperBounds=new ArrayList<>(keys.size());
11     for ( K key : keys) {
12         int upperBound=Integer.MAX_VALUE;
13         for ( Function<K,Integer> upperBoundFunction : upperBounds) {
14             upperBound=Math.min(upperBound,upperBoundFunction
15
16     // Reference code
17     List<Integer> keyUpperBounds=new ArrayList<>(keys.size());
18     for ( K key : keys) {
19         int upperBound=Integer.MAX_VALUE;
20         for ( Function<K,Integer> upperBoundFunction : upperBounds) {
21             upperBound=Math.min(upperBound,upperBoundFunction.apply(key));
22         }
23         keyUpperBounds.add(upperBound);
24     }
25     return keyUpperBounds;
26 }
27

```

- **Explanation:** The erroneous code in this example terminates the code directly at an illegal position, falling under “(formally) incomplete” or “Invalid Generation”. This type of error may have occurred because the code reached the maximum output token length of the LLM.

Table 2. The quantity (Q) and proportion (P) of different hallucination categories in code generated by various LLMs, along with the total number and proportion of each hallucination category (last column). “DS” refers to DeepSeek.

Categories	DS-Coder-1.3B		DS-Coder-7B		CodeLlama-7B		GPT-4		DS-R1		Total	
	Q	P	Q	P	Q	P	Q	P	Q	P	Q	P
Behavior Conflicting	119	9.82%	107	8.83%	127	10.48%	70	5.78%	6	0.50%	429	35.40%
Library/Project	48	3.96%	31	2.56%	45	3.71%	40	3.30%	151	12.46%	315	25.99%
Undefined Variables	44	3.63%	36	2.97%	70	5.78%	29	2.39%	26	2.15%	205	16.91%
Useless Statements (executed without effect)	12	0.99%	24	1.98%	8	0.66%	28	2.31%	1	0.08%	73	6.02%
Algorithm	16	1.32%	13	1.07%	20	1.65%	10	0.83%	1	0.08%	60	4.95%
Data Conflicting	17	1.40%	9	0.74%	18	1.49%	6	0.50%	1	0.08%	51	4.21%
Computer Theory	5	0.41%	7	0.58%	7	0.58%	9	0.74%	0	0.00%	28	2.31%
Fragmented Logics	6	0.50%	1	0.08%	10	0.83%	4	0.33%	0	0.00%	21	1.73%
Mathematics & Natural Science	6	0.50%	3	0.25%	4	0.33%	2	0.17%	2	0.17%	17	1.40%
Useless Statements (unexecuted)	2	0.17%	0	0.00%	4	0.33%	1	0.08%	0	0.00%	7	0.58%
Inconsistent Libraries	0	0.00%	0	0.00%	1	0.08%	2	0.17%	0	0.00%	3	0.25%
Common Sense	1	0.08%	1	0.08%	0	0.00%	1	0.08%	0	0.00%	3	0.25%
Total	276	22.77%	232	19.14%	314	25.91%	202	16.67%	188	15.51%	1212	100.00%

B CODE HALLUCINATION ANALYSIS

B.1 RQ1. Distribution of Code Hallucinations.

We conducted a chi-square test on the results that exhibit obvious differences in RQ1 in order to assess whether these discrepancies are statistically significant. The detailed results are shown as follows, where a, b, c, d represent the values in the 2×2 contingency table.

In addition, to provide more detailed data, we also counted the number and proportion of code hallucinations across different categories, LLMs, and datasets. The results are shown in Table 2 and 3.

B.1.1 Overall Distribution of Hallucinations. To assess whether the observed differences in hallucination occurrence frequency are significant, we present the statistical results in Table 4. We can observe that all the p -values are less than 0.05, indicating that the distribution differences among these types are significant.

Besides, we also investigate whether the difference of the hallucination distribution across LLMs are significant, the results are shown in Table 5. The p -values are less than 0.07, indicating that the distribution differences among these models are significant.

B.1.2 Hallucination Distribution across Benchmarks. In RQ1, we also observe the obvious differences of several hallucination types (Behavior Conflicting, Library/Project) between different benchmarks, thus, we also evaluate whether these differences are significant. As shown in Table 6, all the p -values are less than 0.05, indicating that the differences among these benchmarks are significant.

B.2 RQ2. Hallucination Cause Analysis.

B.2.1 Distribution of Hallucination Causes. The original data on the proportion of causes and impacts for each category of hallucinations is shown in Fig. 3 and Fig. 4.

Table 3. The quantity (Q) and proportion (P) of different hallucination categories in code generated in various datasets.

Categories	CoderEval (Java)		CoderEval (Python)		HumanEval	
	Q	P	Q	P	Q	P
Behavior Conflicting	132	10.89%	198	16.34%	99	8.17%
Library/Project	164	13.53%	146	12.05%	5	0.41%
Undefined Variables	176	14.52%	26	2.15%	3	0.25%
Useless Statements (executed without effect)	67	5.53%	2	0.17%	4	0.33%
Algorithm	41	3.38%	14	1.16%	5	0.41%
Data Conflicting	1	0.08%	14	1.16%	36	2.97%
Computer Theory	1	0.08%	26	2.15%	1	0.08%
Fragmented Logics	2	0.17%	16	1.32%	3	0.25%
Mathematics & Natural Science	0	0.00%	7	0.58%	10	0.83%
Useless Statements (unexecuted)	0	0.00%	0	0.00%	7	0.58%
Inconsistent Libraries	0	0.00%	3	0.25%	0	0.00%
Common Sense	3	0.25%	0	0.00%	0	0.00%
Total	587	48.43%	452	37.29%	173	14.27%

Table 4. Statistical validation results on the distribution of hallucination types among all data. a is the amount of code snippets w/ hallucination Type₁, b is the amount of code snippets w/o hallucination Type₁, c is the amount of code snippets w/ hallucination Type₂, and d is the amount of code snippets w/o hallucination Type₂.

Type ₁	Type ₂	p
Behavior Conflicting	Undefined Variables	8.35×10^{-35}
Library/Project	Computer Theory	4.75×10^{-23}
Library/Project	Algorithm	1.53×10^{-12}

Table 5. Statistical validation results on the distribution of hallucination in different models. a is the amount of hallucinatory code in Model₁, b is the amount of non-hallucinatory code in Model₁, c is the amount of hallucinatory code in Model₂, and d is the amount of non-hallucinatory code in Model₂.

Model ₁	Model ₂	p
DeepSeek-R1	GPT-4	5.54×10^{-2}
GPT-4	DeepSeek-Coder-7B	6.57×10^{-2}
DeepSeek-Coder-7B	DeepSeek-Coder-1.3B	2.12×10^{-2}
DeepSeek-Coder-1.3B	CodeLlama-7B	3.12×10^{-2}

B.2.2 Impact of Prompt Length and Complexity. The heatmap of the proportion of hallucinatory code across different text complexities and prompt lengths, along with the scatter plot depicting the distribution of prompts corresponding to hallucinatory and non-hallucinatory code, generated using raw data, is shown in Figure 2.

Table 6. Statistical validation results on the specific hallucination in different datasets. a is the amount of code w/ hallucination Type $_i$ in Dataset $_1$, b is the amount of code w/o hallucination Type $_i$ in Dataset $_1$, c is the amount of code w/ hallucination Type $_i$ in Dataset $_2$, and d is the amount of code w/o hallucination Type $_i$ in Dataset $_2$.

Type	Dataset $_1$	Dataset $_2$	p
Library/Project	CoderEval (Java)	HumanEval	1.73×10^{-2}
Library/Project	CoderEval (Python)	HumanEval	3.10×10^{-9}
Computer Theory	CoderEval (Python)	CoderEval (Java)	1.84×10^{-12}

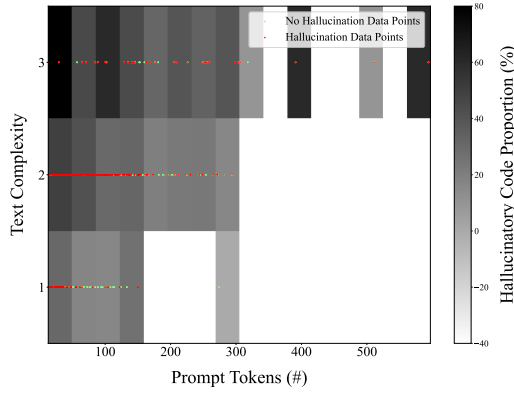


Fig. 2. Heatmap of the proportion of hallucinatory code across different text complexity and length of prompts, along with a scatter plot of the distribution of prompts corresponding to hallucinatory and non-hallucinatory code, generated using raw data. The darker the color, the higher the proportion of hallucinatory code, while the white areas (i.e., ~40%) represent regions with no data points.

We performed a p-test on some correlations between prompt length, complexity, and the proportion of hallucinatory code to support our qualitative analysis in the main text. The results are shown in Table 7.

Table 7. Statistical validation results on some correlations between prompt length, complexity, and the proportion of hallucinatory code.

Regression Model	Tested Variable	p
proportion ~length	length	1.95×10^{-1}
complexity ~length	length	1.50×10^{-4}
proportion ~complexity	complexity	1.31×10^{-2}

The two p-values less than 0.05 demonstrate significant correlations between the corresponding variables, while the p-value greater than 0.05 between proportion and length indicates that there is no significant overall correlation between them (as the proportion first decreases and then increases). These results are fully consistent with our qualitative analysis.

C DISCUSSION

C.1 Hallucination Mitigation Exploration

Here are two supplementary examples of hallucination mitigation:

I. Using *chain-of-thought prompting to mitigate useless statements hallucination*

i. Original

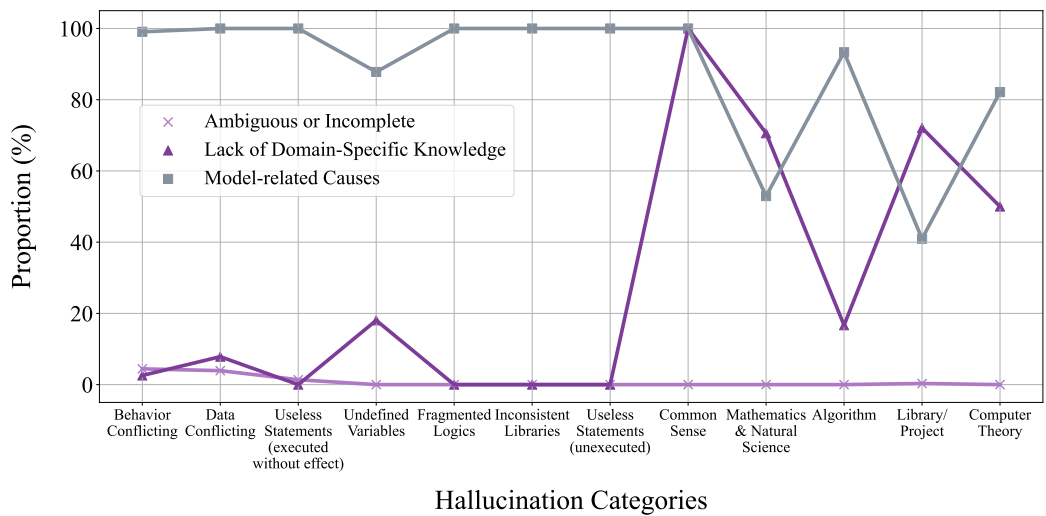


Fig. 3. Causes of various categories of hallucinations.

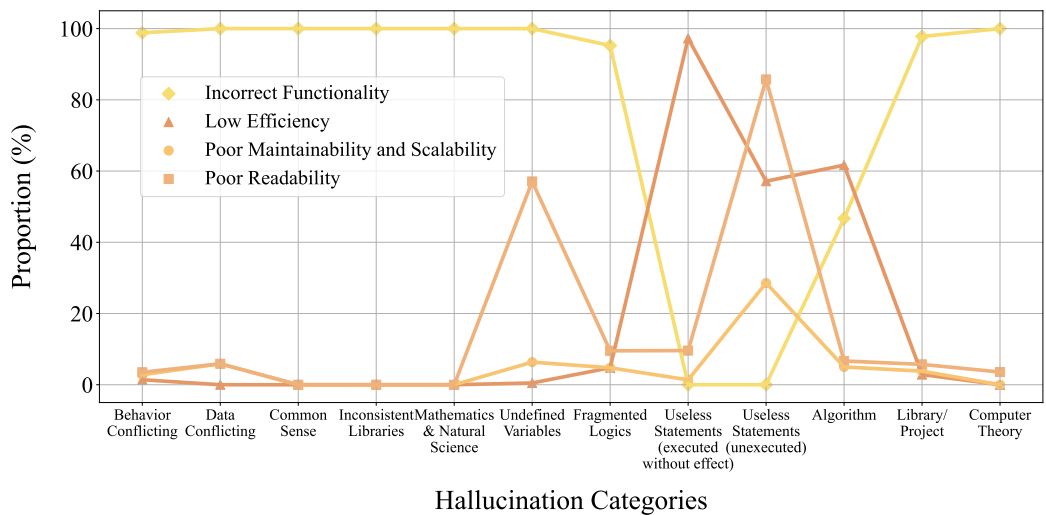


Fig. 4. Impacts of various categories of hallucinations.

• User:

Please try to directly implement this function according to its description:

```
1 def minSubArraySum(nums):
2     """
3     Given an array of integers nums, find the minimum sum
4     of any non-empty sub-array of nums.
5     Example
6     minSubArraySum([2, 3, 4, 1, 2, 4]) == 1
7     minSubArraySum([-1, -2, -3]) == -6
```

8 11 10 11

- DeepSeek-Coder-1.3B:

Here is a Python solution using a sliding window approach:

```

1 def minSubArraySum(nums):
2     if not nums:
3         return 0
4
5     min_sum = float('inf')
6     window_sum = 0
7
8     for i in range(len(nums)):
9         window_sum += nums[i]
10        if window_sum < min_sum:
11            min_sum = window_sum
12        window_sum -= nums[i - len(nums) + 1] if \
13            i >= len(nums) else 0
14
15    return min_sum

```

This function works by iterating over the input array, and for each index, it adds the current number to a running total ('window_sum'). If 'window_sum' is less than 'min_sum', it updates 'min_sum'. Then it subtracts the number at the left end of the window ('nums[i - len(nums) + 1]') and adds the number at the right end of the window ('nums[i]'). This ensures that the window always contains the smallest subarray sum.

The time complexity of this function is $O(n)$, where n is the length of the input array, and the space complexity is $O(1)$, as it only uses a constant amount of space to store the variables 'min_sum' and 'window_sum'.

ii. Chain-of-thought

- User:

```

1 def minSubArraySum(nums):
2     """
3     Given an array of integers nums, find the minimum sum
4     of any non-empty sub-array of nums.
5     Example
6     minSubArraySum([2, 3, 4, 1, 2, 4]) == 1
7     minSubArraySum([-1, -2, -3]) == -6
8     """

```

This is a Python function that needs to be completed. Please provide your steps to solve it.

Let's think step by step:

1.

- DeepSeek-Coder-1.3B:

1. Initialize a variable 'min_sum' to be the first element of the array.
2. Initialize a variable 'current_sum' to be the first element of the array.
3. Iterate over the array starting from the second element.

4. For each element, add it to 'current_sum' and check if 'current_sum' is less than 'min_sum'.
5. If 'current_sum' is less than 'min_sum', update 'min_sum' to be 'current_sum'.
6. Continue this process until you've iterated over the entire array.
7. Return 'min_sum'.

Here is the Python code for the function:

```

1 def minSubArraySum(nums):
2     min_sum = nums[0]
3     current_sum = nums[0]
4     for i in range(1, len(nums)):
5         current_sum = min(current_sum + nums[i], nums[i])
6         min_sum = min(min_sum, current_sum)
7     return min_sum

```

This function works by keeping track of the minimum sum of a subarray ending at each position. It starts by setting the minimum sum to be the first element of the array and the current sum to be the first element of the array. Then it iterates over the array starting from the second element. For each element, it adds the element to the current sum and checks if the current sum is less than the minimum sum. If the current sum is less than the minimum sum, it updates the minimum sum to be the current sum. Finally, it returns the minimum sum.

II. Using related knowledge context to mitigate computer theory knowledge hallucination

i. Original

- User:

Please try to directly implement this function according to its description:

```

1 from typing import List
2
3
4 def string_xor(a: str, b: str) -> str:
5     """ Input are two strings a and b consisting only of
6         1s and 0s. Perform binary XOR on these inputs and
7         return result also as a string.
8     >>> string_xor('010', '110')
9     '100'
10    """

```

- CodeLlama-7B:

Sure, here's an implementation of the 'string_xor' function:

```

1 from typing import List
2
3 def string_xor(a: str, b: str) -> str:
4     result = ""
5     for i in range(len(a)):
6         if a[i] == "1" and b[i] == "1":
7             result += "0"
8         else:
9             result += "1"
10    return result

```

This implementation uses a loop to iterate over the characters in the input strings, and performs a binary XOR operation on each pair of characters. The result is a string of characters, where each character is the XOR of the corresponding characters in the input strings.

For example, if we call the function with the inputs 'a = "010"' and 'b = "110"', the function will return the string "100", which is the binary XOR of the input strings.

Note that this implementation assumes that the input strings are of the same length, and that they only contain the characters '0' and '1'. If the input strings are not of the same length, or if they contain other characters, the function may produce incorrect results.

ii. Augment Prompt with Context

- User:

```
1 from typing import List
2
3
4 def string_xor(a: str, b: str) -> str:
5     """ Input are two strings a and b consisting only of
6         1s and 0s. Perform binary XOR on these inputs and
7         return result also as a string.
8     >>> string_xor('010', '110')
9         '100'
10    """
```

Please complete the function. Here are the related knowledge you can refer to: XOR gate (sometimes EOR, or EXOR and pronounced as Exclusive OR) is a digital logic gate that gives a true (1 or HIGH) output when the number of true inputs is odd. An XOR gate implements an exclusive or () from mathematical logic; that is, a true output results if one, and only one, of the inputs to the gate is true.

- CodeLlama-7B:

Here is the complete implementation of the 'string_xor' function:

```
1 from typing import List
2
3 def string_xor(a: str, b: str) -> str:
4     """ Input are two strings a and b consisting only of
5         1s and 0s. Perform binary XOR on these inputs and
6         return result also as a string.
7     >>> string_xor('010', '110')
8         '100'
9     """
10    result = ''
11    for i in range(len(a)):
12        if a[i] == '1' and b[i] == '1':
13            result += '0'
14        elif a[i] == '0' and b[i] == '0':
15            result += '0'
16        else:
17            result += '1'
18    return result
```

This function takes two strings 'a' and 'b' as input, and performs a bitwise XOR operation on each corresponding bit. The result is a string of the same length as 'a' and 'b', where each bit is the XOR of the corresponding bits in 'a' and 'b'.

For example, if 'a' is '010' and 'b' is '110', the function will return the string '100', since the XOR of '0' and '1' is '1', and the XOR of '1' and '1' is '0'.

Note that this function assumes that the input strings are of the same length, and that they only contain the characters '0' and '1'. If the input strings are not of the same length, or if they contain other characters, the function will raise a 'ValueError'.